VocaGo

Practice Today. Speak Tomorrow.

Nick Michael Rubin

June 21, 2026

Table of Contents

Chapter 1

Analysis

1.1 Description and Requirements

Vocago is a desktop application designed to help users build a strong vocabulary in foreign languages through interactive translation exercises and continuous progress tracking. The application allows users to create personal profiles, manage their own vocabulary entries, and practise them by typing the correct translation of a given word.

The project idea came from the observation that many existing language learning applications primarily focus on grammar, sentence construction, or isolated word memorization. Vocago was created with the goal of improving lexical knowledge, as vocabulary expansion often becomes one of the main challenges once the grammar has been learned.

To address this need, the application allows users to freely build and customize their own vocabulary according to their personal needs and interests, promoting a more flexible and personalized learning experience.

Vocago also emphasizes regular practice and supports learning in both directions between two languages. This bidirectional approach improves both memorization and recognition abilities.

Furthermore, the application allows users to save their vocabulary, monitor their progress over time and continue their learning activities across different study occasions.

Functional Requirements

The application shall allow users to:

- Create, select, edit, and delete user profiles.
- Create and manage personalized vocabulary entries by adding, modifying, and removing words.
- Start learning sessions by practicing vocabulary through typing the correct translation of a given word.
- Study in both directions between two languages.
- Track learning progress and user statistics.
- Provide feedback and notifications related to learning activities and application events.
- Save user data and learning progress between study sessions.

Non-Functional Requirements

The application should satisfy the following non-functional requirements:

- Usability: the graphical interface should provide a simple and intuitive interaction with the system and provide clear feedbacks to the user.
- Maintainability and Extensibility: the software should be organized according to Object Oriented principles and modular design, allowing new learning modes and additional functionalities to be introduced with minimal changes to the system.
- Reliability: user data and learning progress should be preserved, allowing users to continue their learning activities across different study sessions.

1.2 Domain Model

The domain model of Vocago is centered on the activity of learning vocabulary through translation exercises. The main entity is the user profile. Each user has a unique name, a personal vocabulary, and two languages: one language already known by the user and one language to be studied.

The vocabulary is composed of several vocabulary items. Each vocabulary item contains one or more words in the first language and one or more corresponding words in the second language. similarly to an entry in a real dictionary, vocabulary item can support multiple acceptable words or translations.

In addition, each vocabulary item is associated with progress information. Progress includes the number of correct and wrong answers and the current mastery level of the item in each translation direction. The overall results and learning achievements of the user's activity are summarized by the statistics.

The user can start a learning session, during which the application presents translation questions based on the vocabulary items contained in the user's vocabulary. The selection of questions also takes the mastery level of the items into account, so that items with a lower mastery level can be practiced more frequently. Each question is associated with a specific vocabulary item and with a learning direction, which determines whether the requested translation is from the known language to the studied language or in the opposite direction. In this way, both memorization and recognition are tested.

The main entities of the domain and their relationships are shown in Figure 1.1.

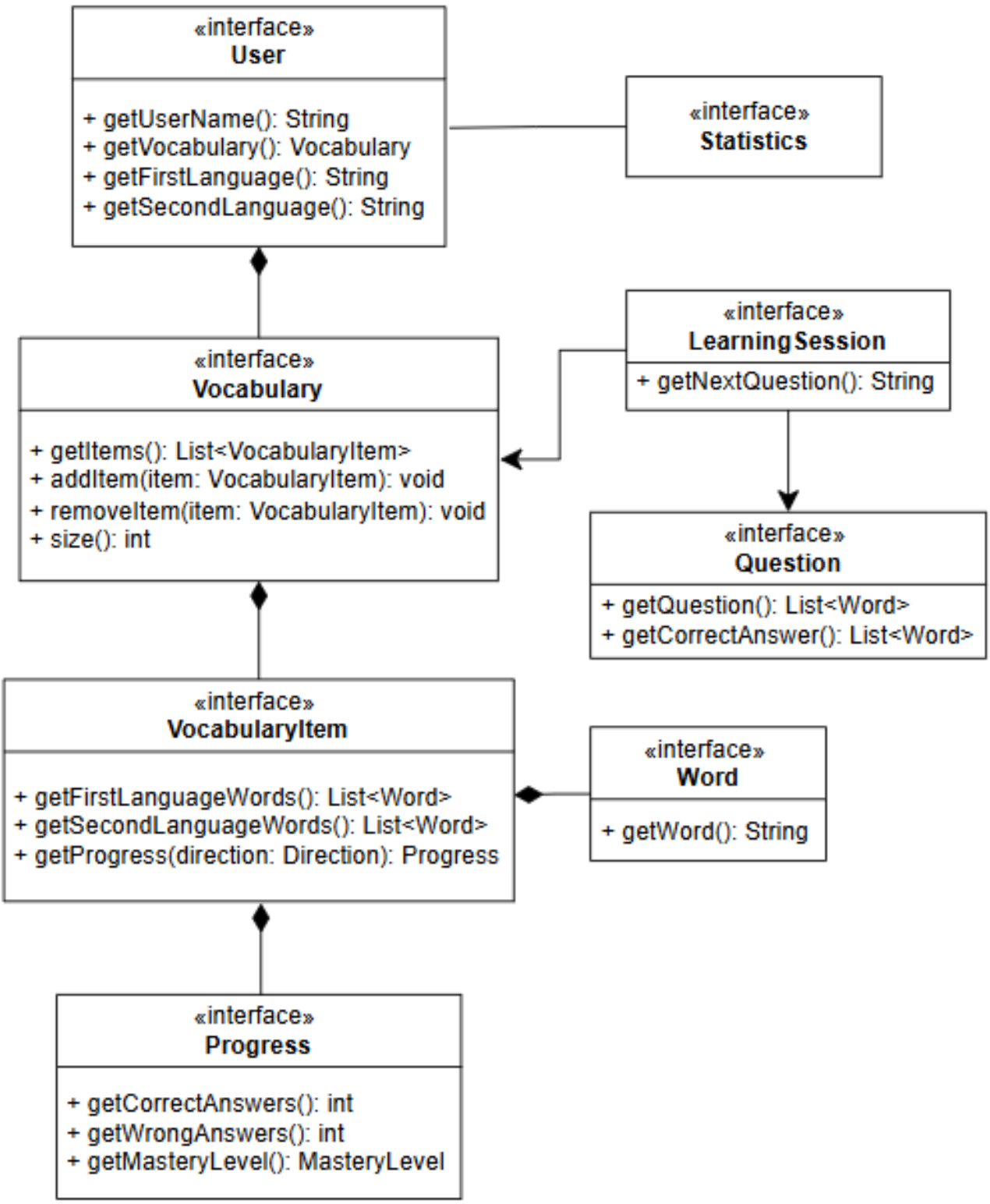

Figure 1.1: Domain model of Vocago and relationships between the main entities

Chapter 2

Design

2.1 Architecture

Vocago is structured according to the Model-View-Controller architectural pattern, with additional services and storage components introduced to keep the system modular. These additional components allow the MVC structure to remain independent from the specific storage solution adopted, while the service layer connects the controller with the model and storage without concentrating too many responsibilities in a single controller component.

The **AppView** interface represents the view of the MVC architecture. It is responsible for displaying the application's graphical interface to the user and collecting user interactions, such as profile selection, vocabulary editing, and learning session inputs.

The **Controller** interface represents the controller of the system and acts as the coordination point between the view and the model. It receives user actions from AppView, invokes the appropriate operations on the application logic and model, and instructs the view to display the updated state. With this design, the view does not directly access the model, and the model does not depend on the graphical interface, so the view can be replaced without changing the controller or the model and the system becomes modular and consistent.

The Model is mainly centered around the **User** interface. It provides access to the user’s vocabulary and learning data. Those domain entities already mentioned in Figure 1.1.

In addition to the main MVC components, the project introduced a service layer and a storage layer. The service layer, represented by **ProfileManager** and **LearningSession** interfaces, connects the controller with the model and persistence, preventing the controller from accumulating too many responsibilities while the storage layer is accessed through **Repository** interfaces.

This additional separation of responsibilities improves modularity, because changes to the graphical interface, learning logic, profile management, or persistence can be introduced with limited impact on the rest of the system.

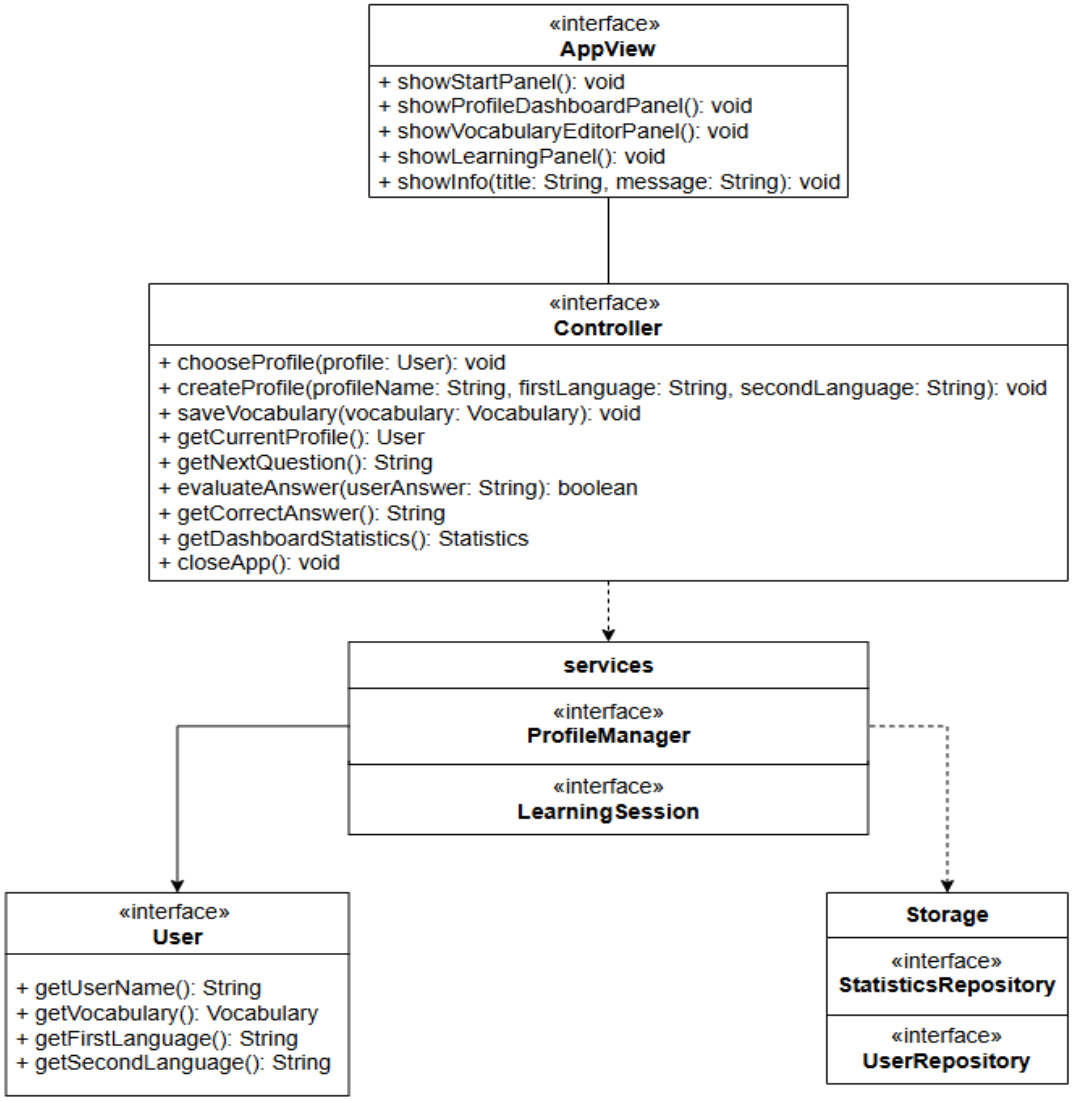

Figure 2.1: General architecture of Vocago, showing the MVC structure extended with services and storage layers.

2.2 Detailed Design

This section presents the main design problems encountered during the development of Vocago and the solutions adopted to address them.

In particular, the discussion will focus on the following problem:

1. Controller organization and responsibility.
2. Profile management and persistence dependency.
3. Learning session management.

1. Controller organization and responsibility

The Problem: The Controller was becoming too large and responsible for too many different tasks, since it had to manage profiles, vocabulary, learning sessions, statistics, navigation, and data saving. This made the controller move away from its main role, which should be coordinating the flow of the system rather than directly handling all application responsibilities. Concentrating all of this logic in a single class risked turning it into a "God class," reducing readability and making every future change touch the same large component.

One of the first options was to create one generic sub-controller holding every operation. This approach would be simple to implement and require less classes. However, it concentrates unrelated responsibilities (profiles, vocabulary, learning, statistics) in one place: every new feature would enlarge the class. This would turn one problem into another and the Single Responsibility Principle would be violated. For these reasons this option was discarded in favour of a decomposition by responsibility.

The Solution: the controller was kept as the main entry point used by the view, but its internal responsibilities were divided among dedicated coordinator components. Profile related operations were delegated to ProfileCoordinator, vocabulary operations to VocabularyCoordinator, learning session operations to LearningCoordinator, and statistics related operations to StatisticsCoordinator. (Figure 2.2)

With this solution, the view can still communicate with a single Controller interface, while the detailed logic of each responsibility is encapsulated inside the corresponding coordinator. ControllerImpl receives requests from the view and forwards them to the appropriate coordinator, instead of directly handling all application logic itself.

This design follows the **Facade** pattern: ControllerImpl provides one simple Controller interface for the view while hiding the four coordinators. The view depends only on the Controller interface and never references the coordinators directly, so the coordinators can be reorganized, split, or extended without any impact on the view.

In addition, this design improves readability and maintainability, because each coordinator has a focused responsibility, and new features can be added to the relevant coordinator without making ControllerImpl more complex.

In the next page is figure 2.2 that shows how the concrete controller delegates different groups of operations to dedicated coordinator components from the whole system.

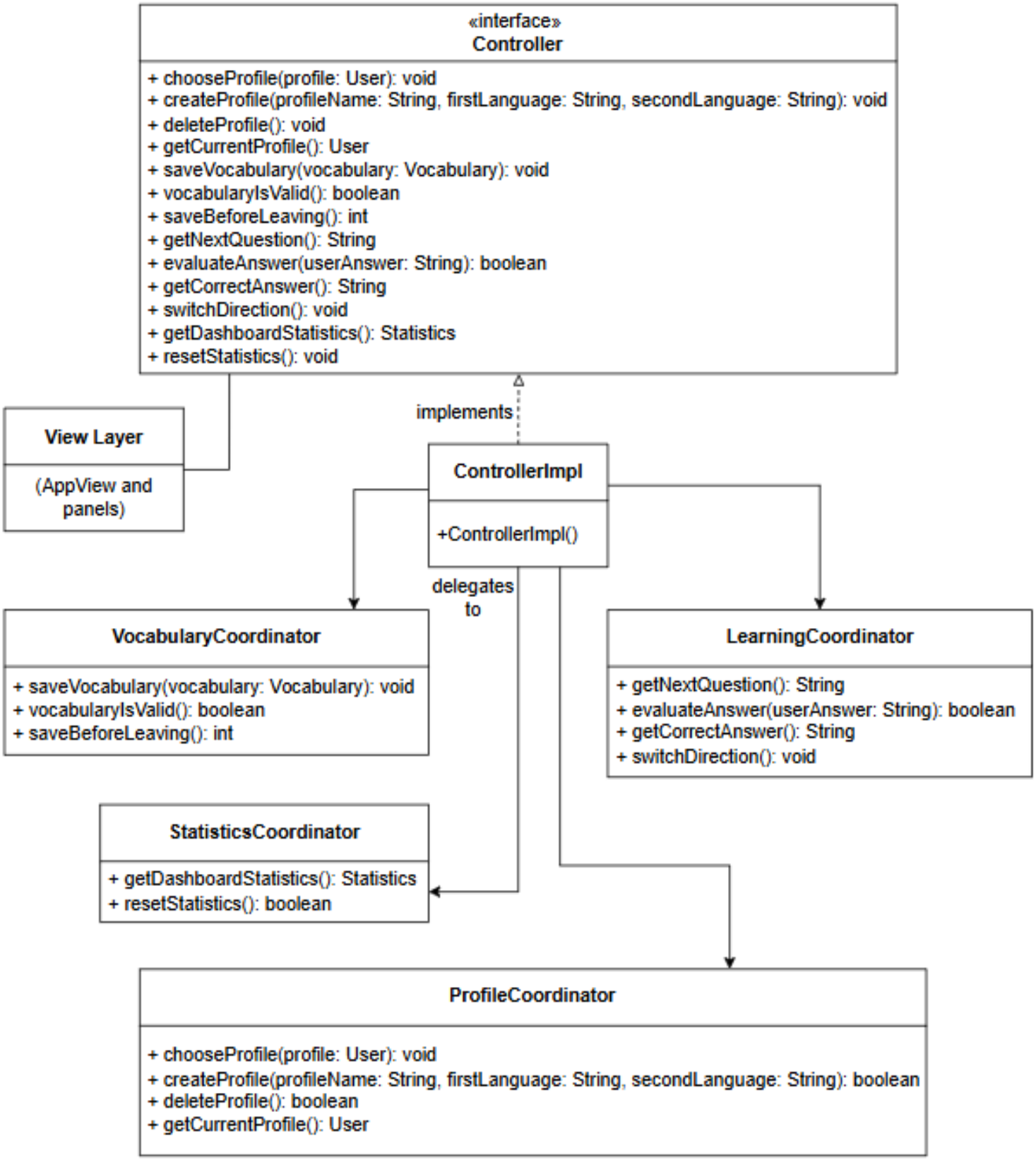

Figure 2.2: Controller responsibility decomposition and delegation to coordinator components.

2. Profile management and persistence dependency

Problem**:** Data is currently stored in CSV and .statistics files, but this could later be replaced by another solution such as an SQL database. If persistence code were placed directly in the controller or domain classes, changing the storage system would require modifications across several parts of the application, making the code harder to maintain. One option was a single persistence class with no repository interfaces: this answers part of the problem, since saving and loading live in their own component instead of the controller or models, but without an interface the application stays coupled to the current file format replacing CSV with a database would mean changing the manager, and a fake storage could not be substituted for testing.

The solution**:** Profile management and persistence were introduced as layers around the MVC structure. Profile logic was placed in a service component, ProfileManager, while saving and loading go through a storage layer using repository interfaces such as UserRepository and StatisticsRepository. This way, the rest of the application will work with any database or storage system.

The storage layer applies the **Repository** pattern: UserRepository and StatisticsRepository are interfaces abstracting data access, realized by the concrete UserCsvStorage and StatisticsFileStorage, so the service layer is unaware of the underlying format. ProfileManagerImpl is also designed for **Dependency Injection**, receiving its repositories through the constructor rather than creating them, so an alternative implementation can be supplied.

In addition, this design makes it more flexible and easier to modify and resolves the problem. If the storage mechanism changes in the future, only the repository implementation needs to be replaced or updated, while the controller, profile management logic, and domain model can remain mostly unchanged.

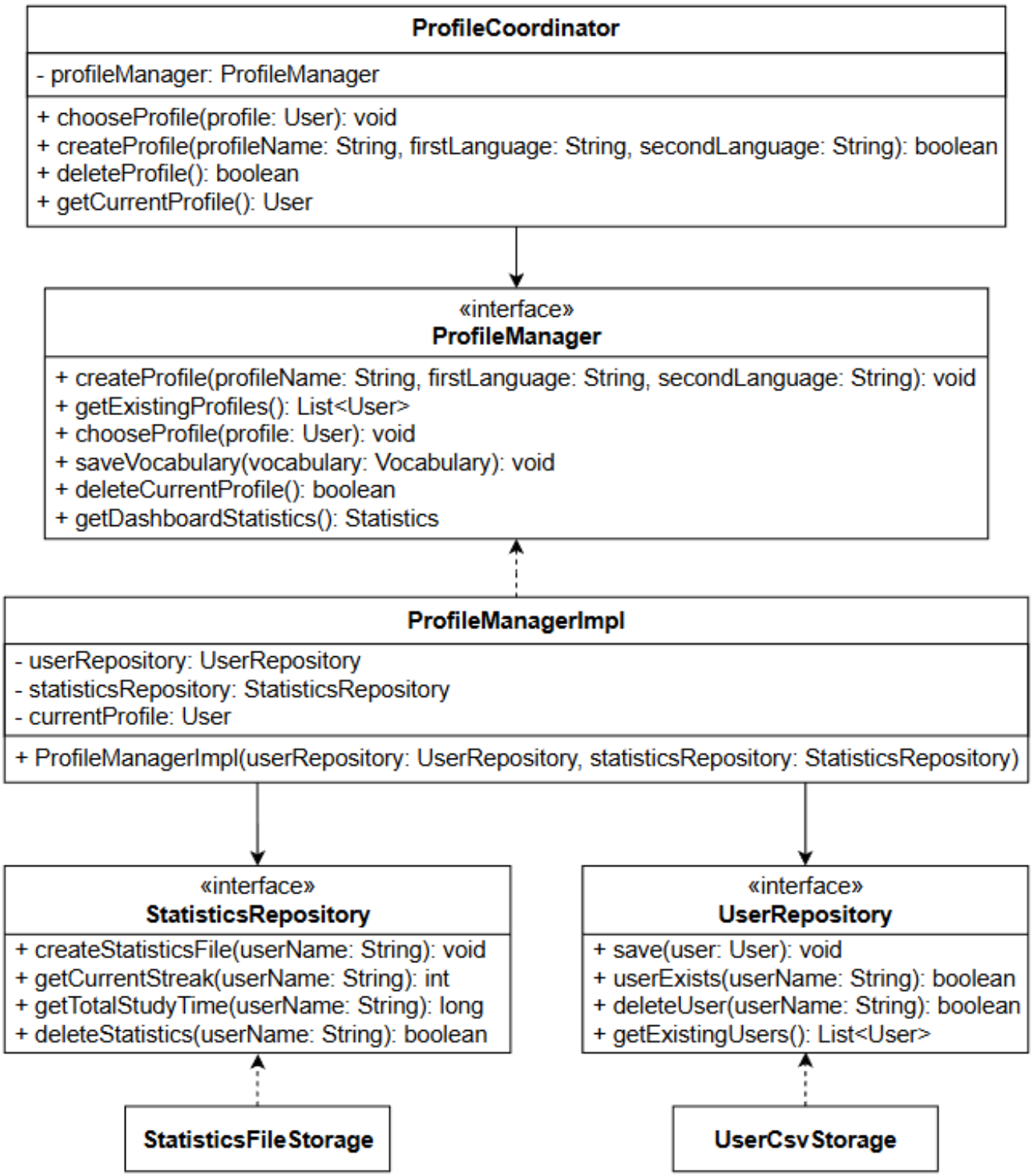

Figure 2.3 continues the controller decomposition shown in Figure 2.2 by focusing on the profile management part. It shows how ProfileManager connects profile logic to storage through repository interfaces, keeping the controller and domain model independent from any chosen storage mechanism

3. Learning session management

The Problem: A learning session needs several connected operations active at once generating questions, evaluating answers, switching direction, tracking the current question and the elapsed time. Placed directly in the controller or view, the flow would be hard to maintain and every new feature would require changes across the system. One option was a single class holding both the session state and logic. pros: simple solution with only one component but the cons: logic could not be changed or reused without touching session state, and the class would keep growing as features are added.

The solution**:** The learning activity was organized around two components. LearningSession keeps the temporary state of the learning session while the actual logic was separated into a LearningEngine. The controller reaches these operations through LearningCoordinator, without managing the internal details.

This design uses the **Strategy** pattern: LearningEngine defines how the next question is chosen and how answers are evaluated, LearningEngineImpl provides the current approach, and LearningSessionImpl simply asks the engine for the next question without knowing how it was selected. Today the engine prioritizes items the user has not yet mastered; other approaches, such as choosing questions from a specific mastery level, could be added later as new LearningEngine implementations without changing the existing session logic.

Separating the engine from the session also keeps the session state independent from the learning logic, making the flow easier to maintain. Features tied to the current activity, like switching direction or timing the session, live inside LearningSession, and the same structure could later support extras such as reviewing incorrectly answered questions.

Figure 2.4 shows how the responsibilities related to the learning flow are organized into dedicated components instead of being implemented directly inside the controller. LearningCoordinator gives access to the learning functionality, LearningSessionImpl manages the temporary state of the current session, and LearningEngine contains the logic for generating questions and evaluating answers.

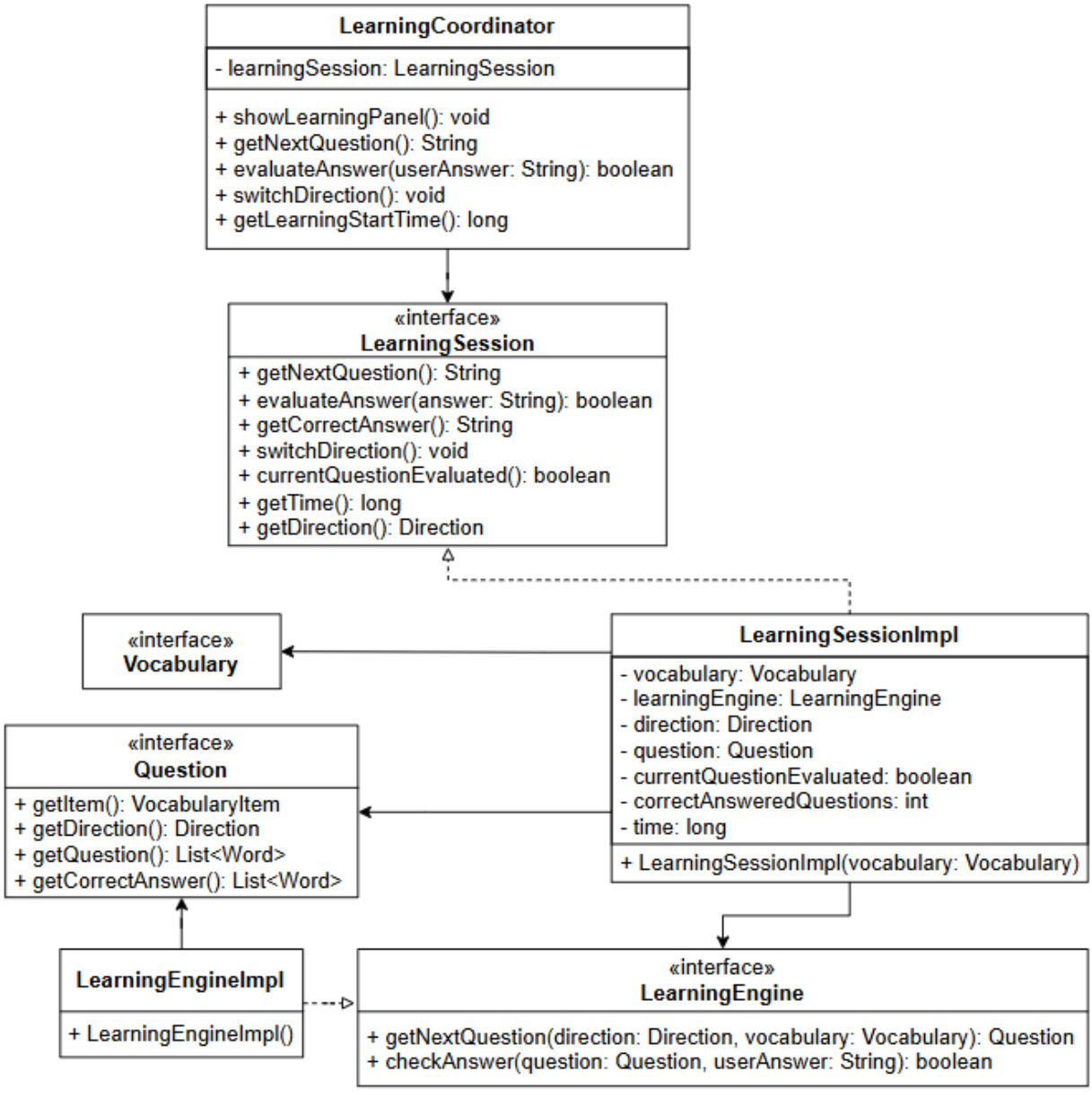

Figure 2.4: Delegation of learning session management from the controller to dedicated learning components.

Chapter 3

Development

3.1 Automated Testing

Automated unit tests were written using JUnit. All tests are fully automatic and require no user interaction. Testing focused on the core model and learning logic, since these form the foundation of the application.
The tested components are:

- MasteryLevel — verifies that the mastery level moves forward on correct answers and backward on wrong ones, stopping correctly at the highest (MASTER) and lowest (BAD) levels.
- Dictionary (Vocabulary) verifies that null items are rejected and that a vocabulary is valid only when it contains at least one complete item.
- Profile (User) — verifies that the constructor trims the profile name, creates an empty vocabulary by default, correctly stores the provided vocabulary and languages, and rejects invalid arguments.
- WordProgress — verifies that an invalid state is rejected, and that correct and wrong answers increment the counters and promote or demote the mastery level at the expected thresholds.
- LearningEngine — verifies that answer checking matches any acceptable answer (trimmed and case-insensitive), that question selection prioritizes new items, and that items missing a translation in one language are skipped.
- Question — verifies that, depending on the learning direction, the correct words are used as prompt and as answer, and that the constructor rejects null arguments.

These tests cover the most important parts of the logic vocabulary validity, mastery progression, question selection and answer checking so that future changes are less likely to cause unexpected behaviour.

3.2 Development Notes

The following advanced features of the Java language and ecosystem were used:

- **Bounded generic method**: [PERMALINK]
- **Generic method**: [PERMALINK]
- **Stream pipelines**: [PERMALINK]
- **Optional with functional chaining:** [PERMALINK], [PERMALINK2]
- **Lambda expressions:** [PERMALINK], [PERMALINK2] (many more all over the view panels)
- **Method references** (e.g. VocabularyItem::isValid): [PERMALINK]

External ideas. The question-selection strategy is based on the general idea behind spaced repetition systems such as SM-2 and the Leitner system [external source link] and [external source link2], which practice weaker items more frequently than well known ones. These algorithms pick what to review based on dates and time intervals, while I based my version on each item's accuracy and mastery level in addition a degree of randomness lets weaker items appear more often while stronger ones still occasionally come up. The adapted implementation can be found here: [PERMALINK].

Chapter 4

Final comments

4.1 Self evaluation and future work

I worked on this project under changing circumstances. It began as a team effort, with another member planned to handle the areas I felt less confident in, namely the database and a large share of the graphical interface, while I focused on areas closer to my strengths. When that member left the group, I had to take over the whole project on my own, including exactly the parts I had counted on not having to build myself. Rather than scaling the project down, I reassessed the situation quickly and decided how to move forward: I reorganized the work around what was realistic for one person, and I took the time to learn the areas I was weaker in instead of avoiding them. Managing the time alone was the hardest part, but I was able to recover and still deliver a complete, working application. Given the circumstances, getting the project to a finished, functioning state on my own is the part I am most satisfied with.
On the design side, the part I'm most satisfied with is the clean separation between the view, controller, service, and storage layers.

I think the weakest part is the implementation itself. In particular, some of the panels were written just to get them working on screen, and the file-storage code relies on manual loops and string handling for reading and parsing the CSV — it works, but it could be cleaner and more robust. With more time I would refactor those parts, and possibly move the storage to an SQL database.

As a foreign student, I built this application partly to help me learn the language myself, so I would genuinely like to keep developing it: adding more learning modes and selection strategies (the LearningEngine interface already makes this easy), improving statistics and progress tracking to give better feedback on what needs practice, and replacing CSV storage with a proper database so the data scales more robustly.

4.2 Difficulties encountered and final comments.

On the technical side, the most time consuming difficulty was setting up and passing the quality assurance tools. Once enabled, they reported a large number of issues at once, and learning which ones to fix and which to suppress with a proper justification took some effort, but it noticeably improved the quality and consistency of the code, even though I would be happy to use less strict tool.

Bundling the application into a single runnable jar that works from an empty configuration was also less straightforward than expected, particularly making sure the image resources were correctly included.

Overall the project was demanding but rewarding, and building a complete working application helped me understand the object oriented concepts of the course far more concretely than the lessons alone.

Appendix A

User guide

Launching the application for the first time, opens the **Create New Profile Panel**, where the user initializes their profile by selecting a target language and creating a profile name.

This action opens the **Profile Dashboard**, a central hub hosting some simple functionalities, like viewing overall statistics, adjusting settings, and switching profiles and the two main ones:

- The **Edit Vocabulary** screen lets you build and change your custom word list. You can add a new row by clicking the button or pressing the Enter key, and you can delete any number of selected rows. There is also a search bar to find words; if you have duplicate words in your list, the search will only show the first match.
  Every word tracks two different skill scores:
  - **Memorization Level:** Your ability to write the word in the language you are studying when presented with a word from a language you already know. (the harder direction)
  - **Recognition Level:** Your ability to write the word in a language you already know when presented with a word from the language you are studying. (the easier direction)

  These mastery levels change automatically based on how well you do during your practice sessions, but you can also click on the grid and change them manually if you fill so. The app uses these levels to decide how often you see a word: as your level for a word gets higher, the app will show it to you less often.

- The **Start Learning** screen is where you actually practice and test yourself. You just type in your translation and press Enter to see if you are right or wrong right away. If you get stuck on a word, you can click **"Skip to Next Word"** to move on without hurting your stats. However, if you type a wrong answer or click **"Reveal Answer"**, the app will penalize you by lowering that word's level so you see it more often. Once you reach your daily goal—which you define in the settings as the

number of words you want to study in a day your streak will be updated to reflect the number of continuous days you have hit this target.

Appendix B

* No laboratory exercises were submitted for this project.